Lesson notes · for you

Sorting, and judging algorithms

Programming · Level 3 · Lesson 4

Time	Materials
20 min	A phone with this lesson downloaded; A pack of cards or numbered paper squares

Learning goals

- Explain bubble, insertion and merge sort.
- Judge algorithms by correctness first, then work.
- Test with edge cases.

Activity: Human sorting

1. Warm-up (3 min): Eight classmates stand in a row with mixed-up number cards. One person counts comparisons on paper.
2. Bubble (7 min): Walk along the row comparing neighbours; swap any pair in the wrong order. Repeat passes until no swaps. Record the comparisons.
3. Insertion (7 min): Mix the row again the same way. Take each person in turn and slide them left past everyone bigger. Record the comparisons.
4. Compare (3 min): Which method needed fewer comparisons? Try again with a row that is nearly sorted.

Write the starting order down so both methods start from the same row: a fair race.

Discussion

- Why count steps instead of seconds?
- When would insertion sort beat merge sort?
- Which edge case would you test first, and why?

Check yourself

- Name three edge cases for a sorting function.

Try it this week
Sort a hand of ten cards twice: once with bubble sort and once with insertion sort. Count the comparisons each time. Which won, and why?

Worksheet

Sorting, and judging algorithms

Programming · Level 3 · Lesson 4

Name _____ Class _____ Date _____

1 **Your sort works on five random lists. Which test should you add first?**

◯ A list of ten thousand items ◯ A sixth random list ◯ An empty list, and one with repeats

2 **Write is_sorted(a): it returns True if every item is less than or equal to the next one, and False otherwise. The tests include the edge cases.**

```
def is_sorted(a):
    # True if each item <= next
    return False
```

Write your program:

3 **Put insertion sort in order: each new item slides left past every bigger one. One line doesn't belong.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

☐ `j = j - 1`

☐ `def insertion_sort(a):`

☐ `a[j - 1], a[j] = a[j], a[j - 1]`

☐ `return a`

☐ `while j > 0 and a[j - 1] < a[j]:`

☐ `while j > 0 and a[j - 1] > a[j]:`

☐ `j = i`

☐ `for i in range(1, len(a)):`

4 **Bubble sort's comparisons grow like n × n. Going from 100 to 1,000 items, about how many times more work does it do?**

◯ 10 times ◯ 100 times ◯ 1,000 times

5 **Match each sort to how it works.**

Bubble sort	• • • •	Sorts the halves, then merges them
Insertion sort	• • • •	Finds the smallest left and swaps it into place
Merge sort	• • • •	Swaps neighbours, pass after pass
Selection sort	• • • •	Slides each new item back into place

✎ **When have you sorted things by hand (cards, books, exam papers)? Which of today's algorithms were you using?**

Check your answers

Sorting, and judging algorithms

Programming · Level 3 · Lesson 4

1 An empty list, and one with repeats

Edge cases are where sorts break: nothing to sort, or equal values.

2

```
def is_sorted(a):
    for i in range(len(a) - 1):
        if a[i] > a[i + 1]:
            return False
    return True
```

One pass over the neighbours is enough. range(len(a) - 1) stops before the last item, so a[i + 1] always exists, and empty and one-item lists are sorted by definition.

3 Top to bottom: 6, 1, 5, 7, X, 4, 3, 2

```
def insertion_sort(a):
    for i in range(1, len(a)):
        j = i
        while j > 0 and a[j - 1] > a[j]:
            a[j - 1], a[j] = a[j], a[j - 1]
            j = j - 1
    return a
```

While the item to the left is bigger, swap and step left.

4 100 times

Ten times the items, each compared with ten times as many: 10 × 10 = 100 times the work.

5

Bubble sort — Swaps neighbours, pass after pass

Insertion sort — Slides each new item back into place

Merge sort — Sorts the halves, then merges them

Selection sort — Finds the smallest left and swaps it into place

Four ways to the same sorted list, doing very different amounts of work.